# Bazy danych  - sprawozdanie z projektu

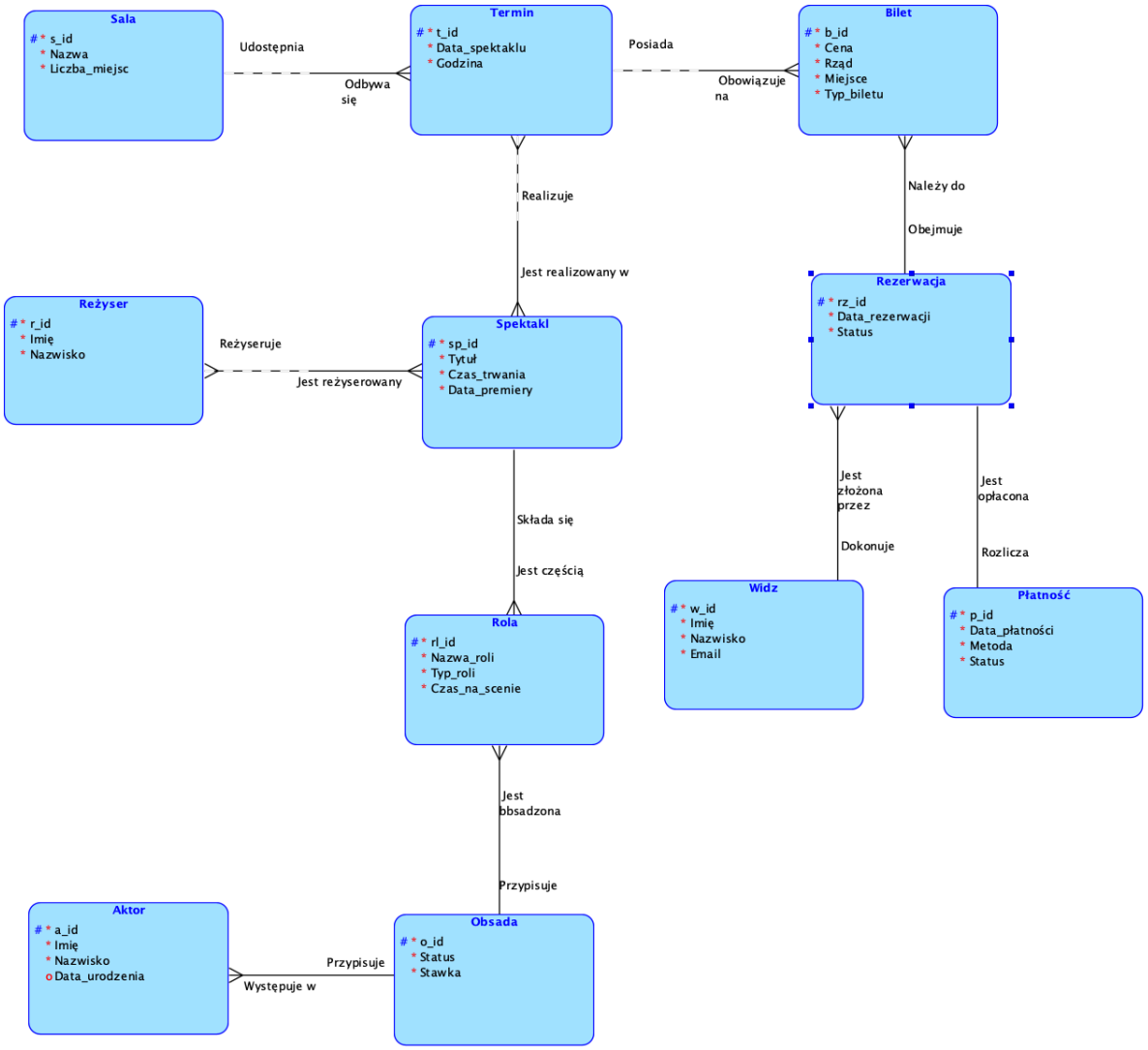
## 1. Wprowadzenie – opis studium przypadku

Realizowanym tematem jest baza danych “Teatr”, która została zaprojektowana w celu kompleksowego wsparcia zarządzania działalnością placówki, obejmując zarówno planowanie spektakli, obsługę widzów, rezerwacje i płatności, jak i analizę frekwencji oraz przychodów.

W teatrze odbywają się różne spektakle w określonych terminach, realizowane w różnych salach. Każdy spektakl posiada przypisanego reżysera, a role są odgrywane przez aktorów z odpowiednią obsadą i wynagrodzeniem. Baza umożliwia dokładne śledzenie terminów spektakli, przypisanie biletów do konkretnych miejsc oraz rezerwacji dokonanych przez widzów.

System pozwala widzom dokonywać rezerwacji biletów i realizować płatności różnymi metodami – od karty, przelewu i BLIKa, po gotówkę – a każda transakcja jest zapisywana z odpowiednim statusem. Dzięki temu teatr może kontrolować zarówno sprzedaż, jak i sytuacje zwrotów lub anulowania rezerwacji.

## 2. Model związków encji




## Opis encji:

1. **Sala** - encja reprezentuje salę teatralną, w której odbywają się spektakle. Zawiera informacje o nazwie sali oraz liczbie dostępnych miejsc.
2. **Termin** - encja określa konkretną datę i godzinę wystawienia spektaklu w danej sali.
3. **Bilet** - encja reprezentuje pojedynczy bilet na określony termin spektaklu, przypisany do konkretnego miejsca.
4. **Reżyser** - encja reprezentuje osobę odpowiedzialną za reżyserię spektaklu.
5. **Spektakl** - encja opisuje przedstawienie teatralne, zawiera informacje o tytule, czasie trwania oraz dacie premiery.

6. **Rezerwacja** - encja opisuje proces rezerwacji biletów przez widza, wraz z datą i statusem rezerwacji (oczekuje, anulowana, potwierdzona)
7. **Rola** - encja opisuje role występujące w spektaklu, wraz z ich typem (pierwszoplanowa, drugoplanowa lub epizodyczna) i czasem przebywania na scenie.
8. **Widz** - encja reprezentuje osobę dokonującą rezerwacji biletów.
9. **Płatność** - encja przechowuje informacje o płatności przypisanej do rezerwacji (status płatności - opłacona, zwrócona oraz metoda płatności - karta, blik, przelew, gotówka)
10. **Aktor** - encja przechowuje dane aktorów biorących udział w spektaklach.
11. **Obsada** - encja pośrednicząca łącząca aktorów z rolami, umożliwiająca określenie statusu obsady (aktywna, zastępcza, gościnna) oraz wynagrodzenia.

## Związki pomiędzy encjami:

1. Sala - Termin

Rodzaj związku: jeden-do-wiele(1:N)

Znaczenie: Jedna sala udostępnia wiele terminów spektakli, natomiast każdy termin odbywa się w jednej, konkretnej sali.

2. Termin - Spektakl

Rodzaj związku: wiele-do-wiele (M:N)

Znaczenie: Jeden termin realizuje wiele spektakli, natomiast jeden spektakl może być realizowany w wielu różnych terminach.

3. Termin -  Bilet

Rodzaj związku: jeden-do-wiele (1:N)

Znaczenie: Jeden termin posiada wiele biletów, natomiast każdy bilet obowiązuje na jeden, konkretny termin.

4. Rezerwacja - Bilet

Rodzaj związku: jeden-do-wiele (1:N)

Znaczenie: Jedna rezerwacja obejmuje jeden lub więcej biletów, natomiast każdy bilet należy do jednej, konkretnej rezerwacji.

5. Widz - Rezerwacja

Rodzaj związku: jeden-do-wiele (1:N)

Znaczenie: Jeden widz dokonuje wielu rezerwacji, natomiast każda rezerwacja jest złożona przez jednego, konkretnego widza.

6. Rezerwacja - Płatność

Rodzaj związku: jeden-do-jeden (1:1)

Znaczenie: Każda rezerwacja jest opłacona jedną płatnością, natomiast każda płatność rozlicza jedną, konkretną rezerwację.

7. Reżyser - Spektakl

Rodzaj związku: wiele-do-wiele (M:N)

Znaczenie: Jeden reżyser reżyseruje wiele spektakli, natomiast każdy spektakl może być reżyserowany przez wiele reżyserów.

8. Spektakl - Rola

Rodzaj związku: jeden-do-wiele(1:N)

Znaczenie: Jeden spektakl składa się z wielu ról, natomiast każda rola jest częścią jednego, konkretnego spektaklu.

9. Obsada - Rola

Rodzaj związku: jeden-do-wiele (1:N)

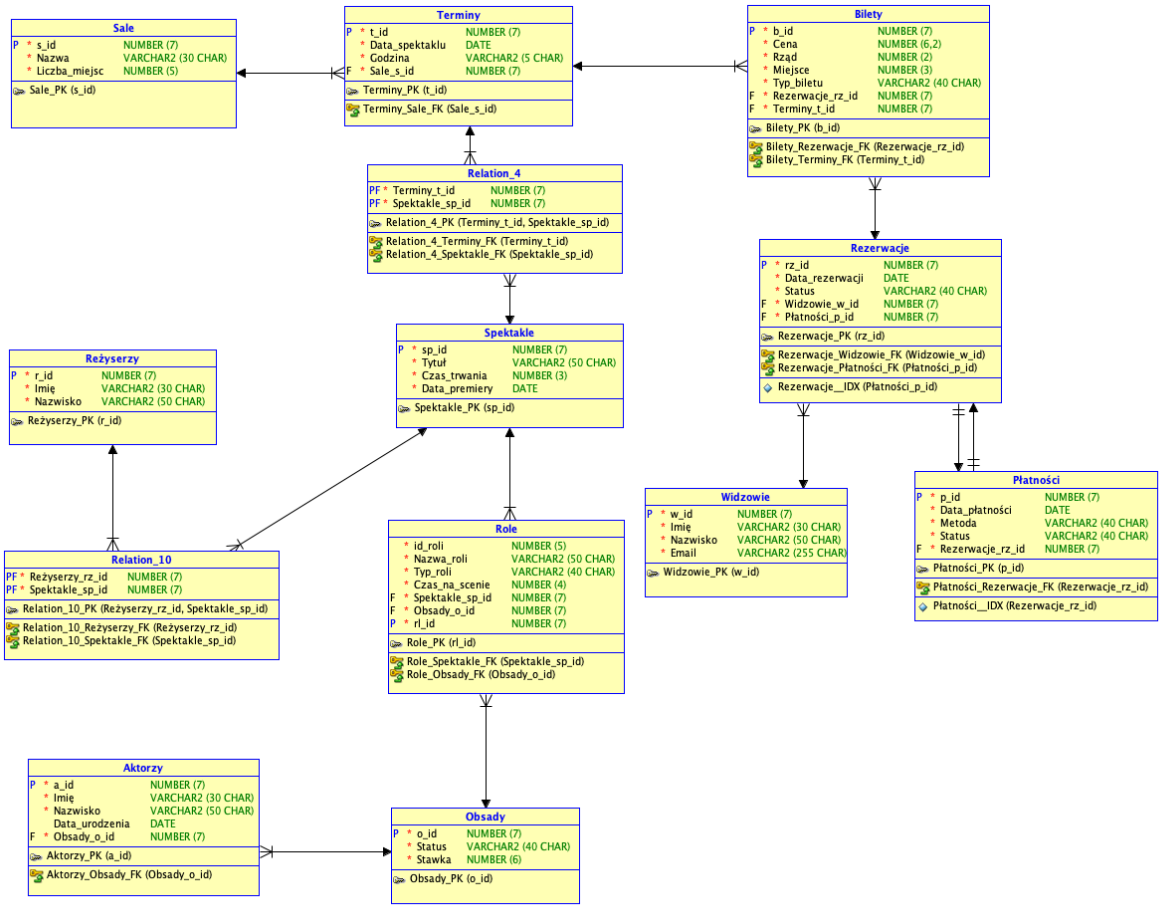
Znaczenie:Jedna rola jest obsadzona przez jedną obsadę, natomiast każda obsada przypisuje wiele ról.

10. Obsada- Aktor

Rodzaj związku: jeden-do-wiele (1:N)

Znaczenie: Jeden aktor występuje w jednej obsadzie, natomiast każda obsada przypisuje wiele aktorów.

# 3. Model relacyjnej bazy danych




## Relacje wiele do wiele (M:N)

Dla relacji typu wiele do wiele zostały wygenerowane dwie tabele łączące:

Relation_4 → Terminy_Spektakle
Tabela realizuje relację M:N pomiędzy encjami Terminy i Spektakle.
Zawiera dwa klucze obce: Terminy_t_id oraz Spektakle_sp_id, które jednocześnie tworzą klucz główny złożony.

Relation_10 → Reżyserzy_Spektakle
Tabela realizuje relację M:N pomiędzy encjami Reżyserzy i Spektakle.
Zawiera klucze obce Reżyserzy_rz_id oraz Spektakle_sp_id, które tworzą klucz główny złożony.

## Relacje jeden do wielu (1:N)

Relacje typu jeden do wielu zostały odwzorowane poprzez umieszczenie klucza obcego w tabeli znajdującej się po stronie „wiele''. Przykłady takich relacji to:

Sale – Terminy (Sale_s_id w tabeli Terminy)

Terminy – Bilety (Terminy_t_id w tabeli Bilety)

Widzowie – Rezerwacje (Widzowie_w_id w tabeli Rezerwacje)

Spektakle – Role (Spektakle_sp_id w tabeli Role)

Role – Obsady (Obsady_o_id w tabeli Role)

Aktorzy – Obsady (Obsady_o_id w tabeli Aktorzy)

## Relacje jeden do jednego (1:1)

Relacja jeden do jednego pomiędzy encjami Rezerwacje i Płatności została odwzorowana przy użyciu kluczy obcych w obu tabelach.

# 4. Schemat bazy - tabele

## 1. Sale

### Atrybuty:

- s_id - Klucz podstawowy (PK)
- Nazwa
- Liczba_miejsc

### DDL:

```
CREATE TABLE Sale
   (
    s_id         NUMBER (7)  NOT NULL ,
    Nazwa        VARCHAR2 (30 CHAR)  NOT NULL ,
    Liczba_miejsc NUMBER (5)  NOT NULL
   )
;

ALTER TABLE Sale
  ADD CONSTRAINT Sale_LiczbaMiejsc_CHK CHECK (Liczba_miejsc > 0);


ALTER TABLE Sale
   ADD CONSTRAINT Sale_PK PRIMARY KEY ( s_id ) ;
```

### DML:

```
INSERT INTO Sale (s_id, Nazwa, Liczba_miejsc) VALUES (2040001, 'Duża Scena', 1200);
INSERT INTO Sale (s_id, Nazwa, Liczba_miejsc) VALUES (2040002, 'Mała Scena', 180);
INSERT INTO Sale (s_id, Nazwa, Liczba_miejsc) VALUES (2040003, 'Scena Kameralna', 90);
INSERT INTO Sale (s_id, Nazwa, Liczba_miejsc) VALUES (2040004, 'Studio', 60);
INSERT INTO Sale (s_id, Nazwa, Liczba_miejsc) VALUES (2040005, 'Sala Nowa', 250);
INSERT INTO Sale (s_id, Nazwa, Liczba_miejsc) VALUES (2040006, 'Sala Eksperymentalna', 140);
INSERT INTO Sale (s_id, Nazwa, Liczba_miejsc) VALUES (2040007, 'Sala Prób', 70);
```

## 2. Terminy

### Atrybuty:

- t_id - Klucz podstawowy (PK)
- Data_spektaklu
- Godzina
- Sale_s_id - Klucz obcy (FK)

### DDL:

```
CREATE TABLE Terminy
   (
    t_id          NUMBER (7)  NOT NULL ,
    Data_spektaklu DATE  NOT NULL ,
    Godzina       VARCHAR2 (5 CHAR)  NOT NULL ,
    Sale_s_id      NUMBER (7)  NOT NULL
   )
;

ALTER TABLE Terminy
   ADD CONSTRAINT Terminy_PK PRIMARY KEY ( t_id ) ;

ALTER TABLE Terminy
   ADD CONSTRAINT Terminy_Sale_FK FOREIGN KEY
   (
    Sale_s_id
   )
   REFERENCES Sale
   (
    s_id
   )
;
```

### DML:

```
INSERT INTO Terminy (t_id, Data_spektaklu, Godzina, Sale_s_id) VALUES (2020001, DATE
'2026-01-10', '18:00', 2040002);

INSERT INTO Terminy (t_id, Data_spektaklu, Godzina, Sale_s_id) VALUES (2020002, DATE
'2026-01-25', '19:00', 2040001);
```

```
INSERT INTO Terminy (t_id, Data_spektaklu, Godzina, Sale_s_id) VALUES (2020003, DATE '2026-02-14', '17:30', 2040005);

INSERT INTO Terminy (t_id, Data_spektaklu, Godzina, Sale_s_id) VALUES (2020004, DATE '2026-03-05', '19:30', 2040003);

INSERT INTO Terminy (t_id, Data_spektaklu, Godzina, Sale_s_id) VALUES (2020005, DATE '2026-04-12', '18:30', 2040002);

INSERT INTO Terminy (t_id, Data_spektaklu, Godzina, Sale_s_id) VALUES (2020006, DATE '2026-06-03', '20:00', 2040005);

INSERT INTO Terminy (t_id, Data_spektaklu, Godzina, Sale_s_id) VALUES (2020007, DATE '2026-08-21', '18:00', 2040006);

INSERT INTO Terminy (t_id, Data_spektaklu, Godzina, Sale_s_id) VALUES (2020008, DATE '2026-09-15', '19:00', 2040001);

INSERT INTO Terminy (t_id, Data_spektaklu, Godzina, Sale_s_id) VALUES (2020009, DATE '2026-10-30', '18:45', 2040002);

INSERT INTO Terminy (t_id, Data_spektaklu, Godzina, Sale_s_id) VALUES (2020010, DATE '2026-12-05', '19:15', 2040005);
```

## 3. Bilety

### Atrybuty:

- b_id - Klucz podstawowy (PK)
- Cena
- Rząd
- Miejsce
- Typ_biletu
- Rezerwacje_rz_id - Klucz obcy (FK)
- Terminy_t_id - Klucz obcy (FK)

### DDL:

```
CREATE TABLE Bilety
  (
   b_id            NUMBER (7)  NOT NULL ,
   Cena            NUMBER (6,2)  NOT NULL ,
   Rząd            NUMBER (2)  NOT NULL ,
   Miejsce         NUMBER (3)  NOT NULL ,
   Typ_biletu      VARCHAR2 (40 CHAR)  NOT NULL ,
   Rezerwacje_rz_id NUMBER (7)  NOT NULL ,
   Terminy_t_id    NUMBER (7)  NOT NULL
  )
```

```
;

ALTER TABLE Bilety
  ADD CONSTRAINT Bilety_Typ_CHK
  CHECK (Typ_biletu IN ('Normalny', 'Ulgowy', 'VIP'));

ALTER TABLE Bilety
  ADD CONSTRAINT Bilety_Cena_CHK CHECK (Cena > 0);

ALTER TABLE Bilety
  ADD CONSTRAINT Bilety_Rzad_CHK CHECK (Rząd > 0);

ALTER TABLE Bilety
  ADD CONSTRAINT Bilety_Miejsce_CHK CHECK (Miejsce > 0);

ALTER TABLE Bilety
    ADD CONSTRAINT Bilety_PK PRIMARY KEY ( b_id ) ;

ALTER TABLE Bilety
    ADD CONSTRAINT Bilety_Rezerwacje_FK FOREIGN KEY
    (
     Rezerwacje_rz_id
    )
    REFERENCES Rezerwacje
    (
     rz_id
    )
;

ALTER TABLE Bilety
    ADD CONSTRAINT Bilety_Terminy_FK FOREIGN KEY
    (
     Terminy_t_id
    )
    REFERENCES Terminy
    (
     t_id
    )
;
```

## DML:

```
-- Termin 2020002

INSERT INTO Bilety (b_id, Cena, Rząd, Miejsce, Typ_biletu, Rezerwacje_rz_id, Terminy_t_id)
VALUES (3010001, 80, 10,  15, 'Normalny', 3020003, 2020002);

INSERT INTO Bilety (b_id, Cena, Rząd, Miejsce, Typ_biletu, Rezerwacje_rz_id, Terminy_t_id)
VALUES (3010002, 80, 10,  16, 'Normalny', 3020015, 2020002);

INSERT INTO Bilety (b_id, Cena, Rząd, Miejsce, Typ_biletu, Rezerwacje_rz_id, Terminy_t_id)
VALUES (3010003, 50, 12,  40, 'Ulgowy',   3020002, 2020002);

INSERT INTO Bilety (b_id, Cena, Rząd, Miejsce, Typ_biletu, Rezerwacje_rz_id, Terminy_t_id)
VALUES (3010004, 150, 3,    1, 'VIP',      3020002, 2020002);

INSERT INTO Bilety (b_id, Cena, Rząd, Miejsce, Typ_biletu, Rezerwacje_rz_id, Terminy_t_id)
VALUES (3010005, 80, 11,  22, 'Normalny', 3020001, 2020002);

-- Termin 2020001 (Hamlet)

INSERT INTO Bilety VALUES (3010006, 80,  6,  12, 'Normalny', 3020001, 2020001);

INSERT INTO Bilety VALUES (3010007, 80,  6,  13, 'Normalny', 3020001, 2020001);

INSERT INTO Bilety VALUES (3010008, 50,  8,  30, 'Ulgowy',   3020002, 2020001);

INSERT INTO Bilety VALUES (3010009, 150, 2,   5, 'VIP',      3020002, 2020001);

-- Termin 2020003 (Dziady)

INSERT INTO Bilety VALUES (3010010, 80,  9,  14, 'Normalny', 3020013, 2020003);

INSERT INTO Bilety VALUES (3010011, 80,  9,  15, 'Normalny', 3020013, 2020003);

INSERT INTO Bilety VALUES (3010012, 50, 11,  50, 'Ulgowy',   3020014, 2020003);

INSERT INTO Bilety VALUES (3010013, 150, 4,   2, 'VIP',      3020014, 2020003);

INSERT INTO Bilety VALUES (3010014, 80, 10,  22, 'Normalny', 3020004, 2020003);

-- Termin 2020004 (Zemsta)

INSERT INTO Bilety VALUES (3010015, 80,  5,  10, 'Normalny', 3020005, 2020004);

INSERT INTO Bilety VALUES (3010016, 80,  5,  11, 'Normalny', 3020005, 2020004);

INSERT INTO Bilety VALUES (3010017, 50,  7,  18, 'Ulgowy',   3020004, 2020004);

-- Termin 2020005 (Mistrz i Małgorzata)

INSERT INTO Bilety VALUES (3010018, 150, 1,   1, 'VIP',      3020006, 2020005);

INSERT INTO Bilety VALUES (3010019, 80,  6,  20, 'Normalny', 3020006, 2020005);

INSERT INTO Bilety VALUES (3010020, 80,  6,  21, 'Normalny', 3020006, 2020005);

-- Termin 2020006 (Mistrz i Małgorzata)

INSERT INTO Bilety VALUES (3010021, 80,  9,  40, 'Normalny', 3020007, 2020006);

INSERT INTO Bilety VALUES (3010022, 50, 11,  55, 'Ulgowy',   3020007, 2020006);

INSERT INTO Bilety VALUES (3010023, 150, 3,   3, 'VIP',      3020008, 2020006);

-- Termin 2020007 (Wesele)

INSERT INTO Bilety VALUES (3010024, 80,  7,  12, 'Normalny', 3020009, 2020007);
```

```
INSERT INTO Bilety VALUES (3010025, 80,  7,  13, 'Normalny', 3020009, 2020007);
INSERT INTO Bilety VALUES (3010026, 50,  9,  25, 'Ulgowy',   3020010, 2020007);
-- Termin 2020008 (Dziady)
INSERT INTO Bilety VALUES (3010027, 80,  8,  10, 'Normalny', 3020011, 2020008);
INSERT INTO Bilety VALUES (3010028, 150, 2,   2, 'VIP',      3020011, 2020008);
-- Termin 2020009 (Zemsta) j
INSERT INTO Bilety VALUES (3010029, 80,  6,  16, 'Normalny', 3020012, 2020009);
INSERT INTO Bilety VALUES (3010030, 80,  6,  17, 'Normalny', 3020012, 2020009);
-- Termin 2020010 (Wesele)
INSERT INTO Bilety VALUES (3010031, 150, 1,   2, 'VIP',      3020012, 2020010);
INSERT INTO Bilety VALUES (3010032, 80,  8,  30, 'Normalny', 3020008, 2020010);
```

## 4. Rezerwacje

### Atrybuty:

- rz_id - Klucz podstawowy (PK)
- Data_rezerwacji
- Status
- Widzowie_w_id - Klucz obcy (FK)
- Płatności_p_id - Klucz obcy (FK)

### DDL:

```
CREATE TABLE Rezerwacje
   (
    rz_id          NUMBER (7)  NOT NULL ,
    Data_rezerwacji DATE  NOT NULL ,
    Status         VARCHAR2 (40 CHAR)  NOT NULL ,
    Widzowie_w_id   NUMBER (7)  NOT NULL ,
    Płatności_p_id  NUMBER (7)  NOT NULL
   )
;

CREATE UNIQUE INDEX Rezerwacje__IDX ON Rezerwacje
   (
    Płatności_p_id ASC
   )
;
```

```
ALTER TABLE Rezerwacje
  ADD CONSTRAINT Rezerwacje_Status_CHK
  CHECK (Status IN ('Oczekuje', 'Potwierdzona', 'Anulowana'));


ALTER TABLE Rezerwacje
    ADD CONSTRAINT Rezerwacje_PK PRIMARY KEY ( rz_id ) ;

ALTER TABLE Rezerwacje
    ADD CONSTRAINT Rezerwacje_Płatności_FK FOREIGN KEY
    (
     Płatności_p_id
    )
    REFERENCES Płatności
    (
     p_id
    )
;

ALTER TABLE Rezerwacje
    ADD CONSTRAINT Rezerwacje_Widzowie_FK FOREIGN KEY
    (
     Widzowie_w_id
    )
    REFERENCES Widzowie
    (
     w_id
    )
;


ALTER TABLE Rezerwacje DISABLE CONSTRAINT Rezerwacje_Płatności_FK;
ALTER TABLE Rezerwacje ENABLE CONSTRAINT Rezerwacje_Płatności_FK;
```

## DML:

```
INSERT INTO Rezerwacje (rz_id, Data_rezerwacji, Status, Widzowie_w_id, Płatności_p_id)

VALUES (seq_rezerwacje.NEXTVAL, DATE '2026-01-09', 'Potwierdzona', 1010001, 3030001);
```

```
INSERT INTO Rezerwacje (rz_id, Data_rezerwacji, Status, Widzowie_w_id, Płatności_p_id)
VALUES (seq_rezerwacje.NEXTVAL, DATE '2025-12-20', 'Potwierdzona', 1010002, 3030002);
INSERT INTO Rezerwacje (rz_id, Data_rezerwacji, Status, Widzowie_w_id, Płatności_p_id)
VALUES (seq_rezerwacje.NEXTVAL, DATE '2026-01-24', 'Oczekuje', 1010003, 3030003);
INSERT INTO Rezerwacje (rz_id, Data_rezerwacji, Status, Widzowie_w_id, Płatności_p_id)
VALUES (seq_rezerwacje.NEXTVAL, DATE '2026-02-01', 'Potwierdzona', 1010004, 3030004);
INSERT INTO Rezerwacje (rz_id, Data_rezerwacji, Status, Widzowie_w_id, Płatności_p_id)
VALUES (seq_rezerwacje.NEXTVAL, DATE '2026-03-01', 'Potwierdzona', 1010005, 3030005);
INSERT INTO Rezerwacje (rz_id, Data_rezerwacji, Status, Widzowie_w_id, Płatności_p_id)
VALUES (seq_rezerwacje.NEXTVAL, DATE '2026-04-11', 'Potwierdzona', 1010006, 3030006);
INSERT INTO Rezerwacje (rz_id, Data_rezerwacji, Status, Widzowie_w_id, Płatności_p_id)
VALUES (seq_rezerwacje.NEXTVAL, DATE '2026-05-20', 'Potwierdzona', 1010007, 3030007);
INSERT INTO Rezerwacje (rz_id, Data_rezerwacji, Status, Widzowie_w_id, Płatności_p_id)
VALUES (seq_rezerwacje.NEXTVAL, DATE '2026-06-01', 'Potwierdzona', 1010008, 3030008);
INSERT INTO Rezerwacje (rz_id, Data_rezerwacji, Status, Widzowie_w_id, Płatności_p_id)
VALUES (seq_rezerwacje.NEXTVAL, DATE '2026-08-20', 'Potwierdzona', 1010009, 3030009);
INSERT INTO Rezerwacje (rz_id, Data_rezerwacji, Status, Widzowie_w_id, Płatności_p_id)
VALUES (seq_rezerwacje.NEXTVAL, DATE '2026-09-01', 'Anulowana', 1010010, 3030010);
INSERT INTO Rezerwacje (rz_id, Data_rezerwacji, Status, Widzowie_w_id, Płatności_p_id)
VALUES (seq_rezerwacje.NEXTVAL, DATE '2026-10-10', 'Potwierdzona', 1010011, 3030011);
INSERT INTO Rezerwacje (rz_id, Data_rezerwacji, Status, Widzowie_w_id, Płatności_p_id)
VALUES (seq_rezerwacje.NEXTVAL, DATE '2026-11-20', 'Potwierdzona', 1010012, 3030012);
INSERT INTO Rezerwacje (rz_id, Data_rezerwacji, Status, Widzowie_w_id, Płatności_p_id)
VALUES (seq_rezerwacje.NEXTVAL, DATE '2026-02-10', 'Potwierdzona', 1010013, 3030013);
INSERT INTO Rezerwacje (rz_id, Data_rezerwacji, Status, Widzowie_w_id, Płatności_p_id)
VALUES (seq_rezerwacje.NEXTVAL, DATE '2026-02-13', 'Potwierdzona', 1010014, 3030014);
INSERT INTO Rezerwacje (rz_id, Data_rezerwacji, Status, Widzowie_w_id, Płatności_p_id)
VALUES (seq_rezerwacje.NEXTVAL, DATE '2026-01-25', 'Potwierdzona', 1010015, 3030015);
```

## 5. Widzowie

### Atrybuty:

- w_id - Klucz podstawowy (PK)
- Imię
- Nazwisko
- Email

### DDL:

```
CREATE TABLE Widzowie
   (
    w_id     NUMBER (7)  NOT NULL ,
    Imię     VARCHAR2 (30 CHAR)  NOT NULL ,
    Nazwisko VARCHAR2 (50 CHAR)  NOT NULL ,
    Email    VARCHAR2 (255 CHAR)  NOT NULL
   )
;

ALTER TABLE Widzowie
   ADD CONSTRAINT Widzowie_PK PRIMARY KEY ( w_id ) ;
```

### DML:

```
INSERT INTO Widzowie (w_id, Imię, Nazwisko, Email)

VALUES (seq_widzowie.NEXTVAL, 'Adam', 'Kowal', 'adam.kowal@mail.pl');

INSERT INTO Widzowie (w_id, Imię, Nazwisko, Email)

VALUES (seq_widzowie.NEXTVAL, 'Beata', 'Nowicka', 'beata.nowicka@mail.pl');

INSERT INTO Widzowie (w_id, Imię, Nazwisko, Email)

VALUES (seq_widzowie.NEXTVAL, 'Celina', 'Borowska', 'celina.borowska@mail.pl');

INSERT INTO Widzowie (w_id, Imię, Nazwisko, Email)

VALUES (seq_widzowie.NEXTVAL, 'Dawid', 'Bielecki', 'dawid.bielecki@mail.pl');

INSERT INTO Widzowie (w_id, Imię, Nazwisko, Email)

VALUES (seq_widzowie.NEXTVAL, 'Eliza', 'Kalinowska', 'eliza.kalinowska@mail.pl');

INSERT INTO Widzowie (w_id, Imię, Nazwisko, Email)
```

VALUES (seq_widzowie.NEXTVAL, 'Filip', 'Zając', 'filip.zajac@mail.pl');

INSERT INTO Widzowie (w_id, Imię, Nazwisko, Email)

VALUES (seq_widzowie.NEXTVAL, 'Gosia', 'Wrona', 'gosia.wrona@mail.pl');

INSERT INTO Widzowie (w_id, Imię, Nazwisko, Email)

VALUES (seq_widzowie.NEXTVAL, 'Hubert', 'Szulc', 'hubert.szulc@mail.pl');

INSERT INTO Widzowie (w_id, Imię, Nazwisko, Email)

VALUES (seq_widzowie.NEXTVAL, 'Iga', 'Kopeć', 'iga.kopec@mail.pl');

INSERT INTO Widzowie (w_id, Imię, Nazwisko, Email)

VALUES (seq_widzowie.NEXTVAL, 'Jakub', 'Czajka', 'jakub.czajka@mail.pl');

INSERT INTO Widzowie (w_id, Imię, Nazwisko, Email)

VALUES (seq_widzowie.NEXTVAL, 'Kinga', 'Pawlak', 'kinga.pawlak@mail.pl');

INSERT INTO Widzowie (w_id, Imię, Nazwisko, Email)

VALUES (seq_widzowie.NEXTVAL, 'Leon', 'Mazur', 'leon.mazur@mail.pl');

INSERT INTO Widzowie (w_id, Imię, Nazwisko, Email)

VALUES (seq_widzowie.NEXTVAL, 'Marta', 'Urban', 'marta.urban@mail.pl');

INSERT INTO Widzowie (w_id, Imię, Nazwisko, Email)

VALUES (seq_widzowie.NEXTVAL, 'Norbert', 'Rybak', 'norbert.rybak@mail.pl');

INSERT INTO Widzowie (w_id, Imię, Nazwisko, Email)

VALUES (seq_widzowie.NEXTVAL, 'Oliwia', 'Sowa', 'oliwia.sowa@mail.pl');

INSERT INTO Widzowie (w_id, Imię, Nazwisko, Email)

VALUES (seq_widzowie.NEXTVAL, 'Patryk', 'Kurek', 'patryk.kurek@mail.pl');

INSERT INTO Widzowie (w_id, Imię, Nazwisko, Email)

VALUES (seq_widzowie.NEXTVAL, 'Roksana', 'Chmiel', 'roksana.chmiel@mail.pl');

INSERT INTO Widzowie (w_id, Imię, Nazwisko, Email)

VALUES (seq_widzowie.NEXTVAL, 'Szymon', 'Madej', 'szymon.madej@mail.pl');

INSERT INTO Widzowie (w_id, Imię, Nazwisko, Email)

VALUES (seq_widzowie.NEXTVAL, 'Tola', 'Król', 'tola.krol@mail.pl');

INSERT INTO Widzowie (w_id, Imię, Nazwisko, Email)

VALUES (seq_widzowie.NEXTVAL, 'Wojtek', 'Wilk', 'wojtek.wilk@mail.pl');

## 6. Płatności

### Atrybuty:

- p_id - Klucz podstawowy (PK)
- Data_platnosci
- Metoda
- Status
- Rezerwacje_rz_id - Klucz obcy (FK)

### DDL:

```
CREATE TABLE Płatności
   (
    p_id            NUMBER (7)  NOT NULL ,
    Data_płatności   DATE  NOT NULL ,
    Metoda          VARCHAR2 (40 CHAR)  NOT NULL ,
    Status          VARCHAR2 (40 CHAR)  NOT NULL ,
    Rezerwacje_rz_id NUMBER (7)  NOT NULL
   )
;
CREATE UNIQUE INDEX Płatności__IDX ON Płatności
   (
    Rezerwacje_rz_id ASC
   )
;

ALTER TABLE Płatności
  ADD CONSTRAINT Platnosci_Status_CHK
  CHECK (Status IN ('Oplacona', 'Zwrocona'));

ALTER TABLE Płatności
  ADD CONSTRAINT Platnosci_Metoda_CHK
  CHECK (Metoda IN ('Karta', 'BLIK', 'Przelew', 'Gotowka'));

ALTER TABLE Płatności
   ADD CONSTRAINT Płatności_PK PRIMARY KEY ( p_id ) ;

ALTER TABLE Płatności
   ADD CONSTRAINT Płatności_Rezerwacje_FK FOREIGN KEY
   (
    Rezerwacje_rz_id
```

```
    )
    REFERENCES Rezerwacje
    (
     rz_id
    )
;

ALTER TABLE Płatności  DISABLE CONSTRAINT Płatności_Rezerwacje_FK;
ALTER TABLE Płatności  ENABLE CONSTRAINT Płatności_Rezerwacje_FK;
```

## DML:

```
INSERT INTO Płatności (p_id, Data_płatności, Metoda, Status, Rezerwacje_rz_id) VALUES
(3030001, DATE '2026-01-09', 'BLIK',     'Oplacona', 3020001);

INSERT INTO Płatności (p_id, Data_płatności, Metoda, Status, Rezerwacje_rz_id) VALUES
(3030002, DATE '2025-12-20', 'Przelew',  'Oplacona', 3020002);

INSERT INTO Płatności (p_id, Data_płatności, Metoda, Status, Rezerwacje_rz_id) VALUES
(3030003, DATE '2026-01-24', 'Karta',    'Oplacona', 3020003);

INSERT INTO Płatności (p_id, Data_płatności, Metoda, Status, Rezerwacje_rz_id) VALUES
(3030004, DATE '2026-02-01', 'Gotowka',  'Oplacona', 3020004);

INSERT INTO Płatności (p_id, Data_płatności, Metoda, Status, Rezerwacje_rz_id) VALUES
(3030005, DATE '2026-03-01', 'Karta',    'Oplacona', 3020005);

INSERT INTO Płatności (p_id, Data_płatności, Metoda, Status, Rezerwacje_rz_id) VALUES
(3030006, DATE '2026-04-11', 'BLIK',     'Oplacona', 3020006);

INSERT INTO Płatności (p_id, Data_płatności, Metoda, Status, Rezerwacje_rz_id) VALUES
(3030007, DATE '2026-05-20', 'Przelew',  'Oplacona', 3020007);

INSERT INTO Płatności (p_id, Data_płatności, Metoda, Status, Rezerwacje_rz_id) VALUES
(3030008, DATE '2026-06-01', 'Karta',    'Oplacona', 3020008);

INSERT INTO Płatności (p_id, Data_płatności, Metoda, Status, Rezerwacje_rz_id) VALUES
(3030009, DATE '2026-08-20', 'BLIK',     'Oplacona', 3020009);

INSERT INTO Płatności (p_id, Data_płatności, Metoda, Status, Rezerwacje_rz_id) VALUES
(3030010, DATE '2026-09-01', 'Karta',    'Zwrocona', 3020010);

INSERT INTO Płatności (p_id, Data_płatności, Metoda, Status, Rezerwacje_rz_id) VALUES
(3030011, DATE '2026-10-10', 'Gotowka',  'Oplacona', 3020011);

INSERT INTO Płatności (p_id, Data_płatności, Metoda, Status, Rezerwacje_rz_id) VALUES
(3030012, DATE '2026-11-20', 'Przelew',  'Oplacona', 3020012);

INSERT INTO Płatności (p_id, Data_płatności, Metoda, Status, Rezerwacje_rz_id) VALUES
(3030013, DATE '2026-02-10', 'Karta',    'Oplacona', 3020013);
```

INSERT INTO Płatności (p_id, Data_płatności, Metoda, Status, Rezerwacje_rz_id) VALUES (3030014, DATE '2026-02-13', 'BLIK',  'Oplacona', 3020014);

INSERT INTO Płatności (p_id, Data_płatności, Metoda, Status, Rezerwacje_rz_id) VALUES (3030015, DATE '2026-01-25', 'Gotowka',  'Oplacona', 3020015);

## 7. Spektakle

### Atrybuty:

- sp_id - Klucz podstawowy (PK)
- Tytuł
- Czas_trwania
- Data_premiery

### DDL:

```
CREATE TABLE Spektakle
   (
    sp_id        NUMBER (7)  NOT NULL ,
    Tytuł        VARCHAR2 (50 CHAR)  NOT NULL ,
    Czas_trwania  NUMBER (3)  NOT NULL ,
    Data_premiery DATE  NOT NULL
   )
;

ALTER TABLE Spektakle
  ADD CONSTRAINT Spektakle_CzasTrwania_CHK CHECK (Czas_trwania > 0);

ALTER TABLE Spektakle
   ADD CONSTRAINT Spektakle_PK PRIMARY KEY ( sp_id ) ;
```

### DML:

INSERT INTO Spektakle (sp_id, Tytuł, Czas_trwania, Data_premiery) VALUES (2010001, 'Hamlet', 165, DATE '2016-01-15');

INSERT INTO Spektakle (sp_id, Tytuł, Czas_trwania, Data_premiery) VALUES (2010002, 'Dziady', 190, DATE '2001-02-20');

INSERT INTO Spektakle (sp_id, Tytuł, Czas_trwania, Data_premiery) VALUES (2010003, 'Zemsta', 110, DATE '2010-03-10');

INSERT INTO Spektakle (sp_id, Tytuł, Czas_trwania, Data_premiery) VALUES (2010004, 'Mistrz i Małgorzata', 150, DATE '2006-04-05');

INSERT INTO Spektakle (sp_id, Tytuł, Czas_trwania, Data_premiery) VALUES (2010005, 'Wesele', 135, DATE '2026-05-12');

## 8. Reżyserzy

### Atrybuty:

- r_id - Klucz podstawowy (PK)
- Imię
- Nazwisko

### DDL:

```
CREATE TABLE Reżyserzy
   (
    r_id     NUMBER (7)  NOT NULL ,
    Imię     VARCHAR2 (30 CHAR)  NOT NULL ,
    Nazwisko VARCHAR2 (50 CHAR)  NOT NULL
   )
;

ALTER TABLE Reżyserzy
   ADD CONSTRAINT Reżyserzy_PK PRIMARY KEY ( r_id ) ;
```

### DML:

```
INSERT INTO Reżyserzy (r_id, Imię, Nazwisko) VALUES (1030001, 'Anna', 'Kowalska');
INSERT INTO Reżyserzy (r_id, Imię, Nazwisko) VALUES (1030002, 'Piotr', 'Nowak');
INSERT INTO Reżyserzy (r_id, Imię, Nazwisko) VALUES (1030003, 'Michał', 'Zieliński');
INSERT INTO Reżyserzy (r_id, Imię, Nazwisko) VALUES (1030004, 'Katarzyna', 'Wiśniewska');
INSERT INTO Reżyserzy (r_id, Imię, Nazwisko) VALUES (1030005, 'Tomasz', 'Wójcik');
INSERT INTO Reżyserzy (r_id, Imię, Nazwisko) VALUES (1030006, 'Julia', 'Kaczmarek');
INSERT INTO Reżyserzy (r_id, Imię, Nazwisko) VALUES (1030007, 'Robert', 'Lewandowski');
```

## 9. Role

### Atrybuty:

- rl_id - Klucz podstawowy (PK)
- Nazwa_roli
- Typ_roli
- Czas_na_scenie
- Spektakle_sp_id - Klucz obcy (FK)
- Obsady_o_id - Klucz obcy (FK)

### DDL:

```
CREATE TABLE Role
   (
    rl_id          NUMBER (7)  NOT NULL ,
   Nazwa_roli      VARCHAR2 (50 CHAR)  NOT NULL ,
   Typ_roli        VARCHAR2 (40 CHAR)  NOT NULL ,
   Czas_na_scenie  NUMBER (4)  NOT NULL ,
   Spektakle_sp_id NUMBER (7)  NOT NULL ,
   Obsady_o_id     NUMBER (7)  NOT NULL
   )
;

ALTER TABLE Role
  ADD CONSTRAINT Role_Typ_CHK
  CHECK (Typ_roli IN ('Pierwszoplanowa', 'Drugoplanowa', 'Epizodyczna'));

ALTER TABLE Role
  ADD CONSTRAINT Role_CzasNaScenie_CHK CHECK (Czas_na_scenie >= 0);

ALTER TABLE Role
   ADD CONSTRAINT Role_PK PRIMARY KEY ( rl_id ) ;


ALTER TABLE Role
   ADD CONSTRAINT Role_Obsady_FK FOREIGN KEY
   (
    Obsady_o_id
   )
   REFERENCES Obsady
```

```
    (
     o_id
    )
;

ALTER TABLE Role
    ADD CONSTRAINT Role_Spektakle_FK FOREIGN KEY
    (
     Spektakle_sp_id
    )
    REFERENCES Spektakle
    (
     sp_id
    )
;
```

## DML:

```
-- Hamlet (2010001)

INSERT INTO Role (rl_id, Nazwa_roli, Typ_roli, Czas_na_scenie, Spektakle_sp_id, Obsady_o_id)
VALUES (2060001, 'Hamlet', 'Pierwszoplanowa', 70, 2010001, 2050001);

INSERT INTO Role (rl_id, Nazwa_roli, Typ_roli, Czas_na_scenie, Spektakle_sp_id, Obsady_o_id)
VALUES (2060002, 'Ofelia', 'Drugoplanowa', 35, 2010001, 2050002);

INSERT INTO Role (rl_id, Nazwa_roli, Typ_roli, Czas_na_scenie, Spektakle_sp_id, Obsady_o_id)
VALUES (2060003, 'Klaudiusz', 'Drugoplanowa', 28, 2010001, 2050003);

INSERT INTO Role (rl_id, Nazwa_roli, Typ_roli, Czas_na_scenie, Spektakle_sp_id, Obsady_o_id)
VALUES (2060004, 'Poloniusz', 'Epizodyczna', 12, 2010001, 2050004);

-- Dziady (2010002)

INSERT INTO Role VALUES (2060005, 'Gustaw-Konrad', 'Pierwszoplanowa', 80, 2010002, 2050001);

INSERT INTO Role VALUES (2060006, 'Ksiądz Piotr',   'Drugoplanowa',    35, 2010002, 2050005);

INSERT INTO Role VALUES (2060007, 'Senator',        'Drugoplanowa',    30, 2010002, 2050003);

INSERT INTO Role VALUES (2060008, 'Widmo',          'Epizodyczna',     10, 2010002, 2050006);

-- Zemsta (2010003)

INSERT INTO Role VALUES (2060009,  'Cześnik',  'Pierwszoplanowa', 45, 2010003, 2050002);

INSERT INTO Role VALUES (2060010, 'Rejent',   'Pierwszoplanowa', 40, 2010003, 2050003);

INSERT INTO Role VALUES (2060011, 'Papkin',   'Drugoplanowa',    25, 2010003, 2050004);

-- Mistrz i Małgorzata (2010004)

INSERT INTO Role VALUES (2060012, 'Mistrz',      'Pierwszoplanowa', 55, 2010004, 2050001);

INSERT INTO Role VALUES (2060013, 'Małgorzata',  'Pierwszoplanowa', 55, 2010004, 2050002);
```

```
INSERT INTO Role VALUES (2060014, 'Woland',      'Drugoplanowa',    30, 2010004, 2050007);
-- Wesele (2010005)
INSERT INTO Role VALUES (2060015, 'Pan Młody', 'Pierwszoplanowa', 40, 2010005, 2050003);
INSERT INTO Role VALUES (2060016, 'Panna Młoda','Drugoplanowa',   35, 2010005, 2050002);
INSERT INTO Role VALUES (2060017, 'Poeta',     'Epizodyczna',     15, 2010005, 2050006);
INSERT INTO Role VALUES (2060018, 'Dziennikarz','Epizodyczna',    12, 2010005, 2050005);
```

## 10. Aktorzy

### Atrybuty:

- a_id - Klucz podstawowy (PK)
- Imie
- Nazwisko
- Data_urodzenia
- Obsady_o_id - Klucz obcy (FK)

### DDL:

```
CREATE TABLE Aktorzy
   (
    a_id          NUMBER (7)  NOT NULL ,
    Imię          VARCHAR2 (30 CHAR)  NOT NULL ,
    Nazwisko      VARCHAR2 (50 CHAR)  NOT NULL ,
    Data_urodzenia DATE ,
    Obsady_o_id    NUMBER (7)  NOT NULL
   )
;

ALTER TABLE Aktorzy
   ADD CONSTRAINT Aktorzy_PK PRIMARY KEY ( a_id ) ;

ALTER TABLE Aktorzy
   ADD CONSTRAINT Aktorzy_Obsady_FK FOREIGN KEY
   (
    Obsady_o_id
   )
   REFERENCES Obsady
   (
    o_id
   )
```

;

### DML:

INSERT INTO Aktorzy (a_id, Imię, Nazwisko, Data_urodzenia, Obsady_o_id) VALUES (1020001, 'Jan', 'Maj', DATE '1992-04-11', 2050001);

INSERT INTO Aktorzy (a_id, Imię, Nazwisko, Data_urodzenia, Obsady_o_id) VALUES (1020002, 'Ewa', 'Lis', DATE '1995-09-03', 2050001);

INSERT INTO Aktorzy (a_id, Imię, Nazwisko, Data_urodzenia, Obsady_o_id) VALUES (1020003, 'Kamil', 'Kruk', DATE '1988-01-27', 2050001);

INSERT INTO Aktorzy (a_id, Imię, Nazwisko, Data_urodzenia, Obsady_o_id) VALUES (1020004, 'Ola', 'Bąk', DATE '1999-06-18', 2050002);

INSERT INTO Aktorzy (a_id, Imię, Nazwisko, Data_urodzenia, Obsady_o_id) VALUES (1020005, 'Paweł', 'Nowicki', DATE '1990-12-02', 2050002);

INSERT INTO Aktorzy (a_id, Imię, Nazwisko, Data_urodzenia, Obsady_o_id) VALUES (1020006, 'Magda', 'Sikora', DATE '1993-03-22', 2050003);

INSERT INTO Aktorzy (a_id, Imię, Nazwisko, Data_urodzenia, Obsady_o_id) VALUES (1020007, 'Tomek', 'Piątek', DATE '1987-07-15', 2050003);

INSERT INTO Aktorzy (a_id, Imię, Nazwisko, Data_urodzenia, Obsady_o_id) VALUES (1020008, 'Natalia', 'Dąbrowska', DATE '1996-11-09', 2050004);

INSERT INTO Aktorzy (a_id, Imię, Nazwisko, Data_urodzenia, Obsady_o_id) VALUES (1020009, 'Marcin', 'Jaworski', DATE '1991-05-30', 2050005);

INSERT INTO Aktorzy (a_id, Imię, Nazwisko, Data_urodzenia, Obsady_o_id) VALUES (1020010, 'Karol', 'Baran', DATE '1985-10-20', 2050005);

## 11. Obsady

### Atrybuty:

- o_id - Klucz podstawowy (PK)
- Status
- Stawka

### DDL:

```
CREATE TABLE Obsady
  (
   o_id   NUMBER (7)  NOT NULL ,
   Status VARCHAR2 (40 CHAR)  NOT NULL ,
   Stawka NUMBER (6)  NOT NULL
  )
```

```
;

ALTER TABLE Obsady
  ADD CONSTRAINT Obsady_Stawka_CHK CHECK (Stawka > 0);

ALTER TABLE Obsady
    ADD CONSTRAINT Obsady_PK PRIMARY KEY ( o_id ) ;
```

### DML:

```
INSERT INTO Obsady (o_id, Status, Stawka) VALUES (2050001, 'Aktywna', 900);
INSERT INTO Obsady (o_id, Status, Stawka) VALUES (2050002, 'Aktywna', 800);
INSERT INTO Obsady (o_id, Status, Stawka) VALUES (2050003, 'Aktywna', 750);
INSERT INTO Obsady (o_id, Status, Stawka) VALUES (2050004, 'Aktywna', 650);
INSERT INTO Obsady (o_id, Status, Stawka) VALUES (2050005, 'Aktywna', 700);
INSERT INTO Obsady (o_id, Status, Stawka) VALUES (2050006, 'Zastepcza', 500);
INSERT INTO Obsady (o_id, Status, Stawka) VALUES (2050007, 'Goscinna', 1200);
```

## 12. Terminy_Spektakle

### Atrybuty:

- Terminy_t_id - Klucz podstawowy (PK)
- Spektakle_sp_id - Klucz podstawowy (PK)

### DDL:

```
CREATE TABLE Relation_4
    (
     Terminy_t_id    NUMBER (7)  NOT NULL ,
     Spektakle_sp_id NUMBER (7)  NOT NULL
    )
;

ALTER TABLE Relation_4
    ADD CONSTRAINT Relation_4_PK PRIMARY KEY ( Terminy_t_id, Spektakle_sp_id ) ;

RENAME Relation_4 TO Terminy_Spektakle;

ALTER TABLE Relation_4
    ADD CONSTRAINT Relation_4_Spektakle_FK FOREIGN KEY
```

```
    (
     Spektakle_sp_id
    )
    REFERENCES Spektakle
    (
     sp_id
    )
;

ALTER TABLE Relation_4
    ADD CONSTRAINT Relation_4_Terminy_FK FOREIGN KEY
    (
     Terminy_t_id
    )
    REFERENCES Terminy
    (
     t_id
    )
;
```

### DML:

```
INSERT INTO Terminy_Spektakle (Terminy_t_id, Spektakle_sp_id) VALUES (2020001, 2010001);
INSERT INTO Terminy_Spektakle (Terminy_t_id, Spektakle_sp_id) VALUES (2020002, 2010001);
INSERT INTO Terminy_Spektakle (Terminy_t_id, Spektakle_sp_id) VALUES (2020003, 2010002);
INSERT INTO Terminy_Spektakle (Terminy_t_id, Spektakle_sp_id) VALUES (2020004, 2010003);
INSERT INTO Terminy_Spektakle (Terminy_t_id, Spektakle_sp_id) VALUES (2020005, 2010004);
INSERT INTO Terminy_Spektakle (Terminy_t_id, Spektakle_sp_id) VALUES (2020006, 2010004);
INSERT INTO Terminy_Spektakle (Terminy_t_id, Spektakle_sp_id) VALUES (2020007, 2010005);
INSERT INTO Terminy_Spektakle (Terminy_t_id, Spektakle_sp_id) VALUES (2020008, 2010002);
INSERT INTO Terminy_Spektakle (Terminy_t_id, Spektakle_sp_id) VALUES (2020009, 2010003);
INSERT INTO Terminy_Spektakle (Terminy_t_id, Spektakle_sp_id) VALUES (2020010, 2010005);
```

## 13. Rezyserzy_Spektakle

### Atrybuty:

- Rezyserzy_rz_id - Klucz podstawowy (PK)
- Spektakle_sp_id - Klucz podstawowy (PK)

## DDL:

```
CREATE TABLE Relation_10
   (
    Reżyserzy_rz_id NUMBER (7)  NOT NULL ,
    Spektakle_sp_id NUMBER (7)  NOT NULL
   )
;

ALTER TABLE Relation_10
   ADD CONSTRAINT Relation_10_PK PRIMARY KEY ( Reżyserzy_rz_id, Spektakle_sp_id ) ;

RENAME Relation_10 TO Rezyserzy_Spektakle;

ALTER TABLE Relation_10
   ADD CONSTRAINT Relation_10_Reżyserzy_FK FOREIGN KEY
   (
    Reżyserzy_rz_id
   )
   REFERENCES Reżyserzy
   (
    r_id
   )
;

ALTER TABLE Relation_10
   ADD CONSTRAINT Relation_10_Spektakle_FK FOREIGN KEY
   (
    Spektakle_sp_id
   )
   REFERENCES Spektakle
   (
    sp_id
   )
;
```

## DML:

```
INSERT INTO Rezyserzy_Spektakle (Reżyserzy_rz_id, Spektakle_sp_id) VALUES (1030001,
2010001);
```

INSERT INTO Rezyserzy_Spektakle (Reżyserzy_rz_id, Spektakle_sp_id) VALUES (1030002, 2010002);

INSERT INTO Rezyserzy_Spektakle (Reżyserzy_rz_id, Spektakle_sp_id) VALUES (1030003, 2010003);

INSERT INTO Rezyserzy_Spektakle (Reżyserzy_rz_id, Spektakle_sp_id) VALUES (1030004, 2010004);

INSERT INTO Rezyserzy_Spektakle (Reżyserzy_rz_id, Spektakle_sp_id) VALUES (1030005, 2010005);

INSERT INTO Rezyserzy_Spektakle (Reżyserzy_rz_id, Spektakle_sp_id) VALUES (1030006, 2010002);

INSERT INTO Rezyserzy_Spektakle (Reżyserzy_rz_id, Spektakle_sp_id) VALUES (1030007, 2010004);

# 5. Przypadki użycia bazy danych

**- Przypadek użycia 1**. Wyświetlanie repertuaru od określonej daty

```
SELECT t.Data_spektaklu, t.Godzina, s.Nazwa sala, sp.Tytuł tytul, r.Imię || ' ' || r.Nazwisko rezyser
FROM Terminy t JOIN Sale s ON s.s_id = t.Sale_s_id
   JOIN Terminy_Spektakle ts ON ts.Terminy_t_id = t.t_id
   JOIN Spektakle sp ON sp.sp_id = ts.Spektakle_sp_id
   JOIN Rezyserzy_Spektakle rs ON rs.Spektakle_sp_id = sp.sp_id
   JOIN Reżyserzy r ON r.r_id = rs.Reżyserzy_rz_id
WHERE t.Data_spektaklu >= DATE '2026-01-01'
ORDER BY t.Data_spektaklu, t.Godzina;
```

Opis: Przypadek ten umożliwia wyświetlenie repertuaru od dnia 01.01.2026, wraz z informacjami o dacie i godzinie spektaklu, sali, tytule spektaklu oraz jego reżyserze.

| | DATA_SPEKTAKLU | GODZINA | SALA | TYTUL | REZYSER |
|---|---|---|---|---|---|
| 1 | 26/01/10 | 18:00 | Mała Scena | Hamlet | Anna Kowalska |
| 2 | 26/01/25 | 19:00 | Duża Scena | Hamlet | Anna Kowalska |
| 3 | 26/02/14 | 17:30 | Sala Nowa | Dziady | Piotr Nowak |
| 4 | 26/02/14 | 17:30 | Sala Nowa | Dziady | Julia Kaczmarek |
| 5 | 26/03/05 | 19:30 | Scena Kameralna | Zemsta | Michał Zieliński |
| 6 | 26/04/12 | 18:30 | Mała Scena | Mistrz i Małgorzata | Katarzyna Wiśniewska |
| 7 | 26/04/12 | 18:30 | Mała Scena | Mistrz i Małgorzata | Robert Lewandowski |
| 8 | 26/06/03 | 20:00 | Sala Nowa | Mistrz i Małgorzata | Robert Lewandowski |
| 9 | 26/06/03 | 20:00 | Sala Nowa | Mistrz i Małgorzata | Katarzyna Wiśniewska |
| 10 | 26/08/21 | 18:00 | Sala Eksperymentalna | Wesele | Tomasz Wójcik |
| 11 | 26/09/15 | 19:00 | Duża Scena | Dziady | Piotr Nowak |
| 12 | 26/09/15 | 19:00 | Duża Scena | Dziady | Julia Kaczmarek |
| 13 | 26/10/30 | 18:45 | Mała Scena | Zemsta | Michał Zieliński |
| 14 | 26/12/05 | 19:15 | Sala Nowa | Wesele | Tomasz Wójcik |

**- Przypadek użycia 2**. Analiza obłożenia sal

```
SELECT t.Data_spektaklu, t.Godzina, s.Nazwa sala, s.Liczba_miejsc, COUNT(b.b_id) sprzedane,
ROUND(COUNT(b.b_id) / s.Liczba_miejsc * 100, 2) oblozenie_proc
FROM Terminy t
JOIN Sale s ON s.s_id = t.Sale_s_id
LEFT JOIN Bilety b ON b.Terminy_t_id = t.t_id
GROUP BY t.Data_spektaklu, t.Godzina, s.Nazwa, s.Liczba_miejsc
HAVING COUNT(b.b_id) <= 0.01 * s.Liczba_miejsc
ORDER BY oblozenie_proc DESC;
```

Opis: Zapytanie oblicza obłożenie sal dla poszczególnych terminów spektakli. Wybierane są te przypadki, gdzie liczba sprzedanych biletów stanowi mniej niż 1% pojemności danej sali

| | DATA_SPEKTAKLU | GODZINA | SALA | LICZBA_MIEJSC | SPRZEDANE | OBLOZENIE_PROC |
|---|---|---|---|---|---|---|
| 1 | 26/12/05 | 19:15 | Sala Nowa | 250 | 2 | 0,8 |
| 2 | 26/01/25 | 19:00 | Duża Scena | 1200 | 5 | 0,42 |
| 3 | 26/09/15 | 19:00 | Duża Scena | 1200 | 2 | 0,17 |

**- Przypadek użycia 3**. Zestawienie sprzedaży biletów i przychodów dla terminów

```
SELECT t.t_id, t.Data_spektaklu, t.Godzina, s.Nazwa sala, NVL(x.liczba_biletow, 0) liczba_biletow,
NVL(x.przychod, 0) przychod
FROM Terminy t
JOIN Sale s ON s.s_id = t.Sale_s_id
LEFT JOIN (
  SELECT
    Terminy_t_id,
    COUNT(*) liczba_biletow,
    SUM(Cena) przychod
  FROM Bilety
  GROUP BY Terminy_t_id
) x ON x.Terminy_t_id = t.t_id
ORDER BY t.Data_spektaklu, t.Godzina;
```

Opis: Dla każdego terminu spektaklu prezentowana jest liczba sprzedanych biletów oraz wygenerowany przychód.

| | T_ID | DATA_SPEKTAKLU | GODZINA | SALA | LICZBA_BILETOW | PRZYCHOD |
|---|---|---|---|---|---|---|
| 1 | 2020001 | 26/01/10 | 18:00 | Mała Scena | 4 | 360 |
| 2 | 2020002 | 26/01/25 | 19:00 | Duża Scena | 5 | 440 |
| 3 | 2020003 | 26/02/14 | 17:30 | Sala Nowa | 5 | 440 |
| 4 | 2020004 | 26/03/05 | 19:30 | Scena Kameralna | 3 | 210 |
| 5 | 2020005 | 26/04/12 | 18:30 | Mała Scena | 3 | 310 |
| 6 | 2020006 | 26/06/03 | 20:00 | Sala Nowa | 3 | 280 |
| 7 | 2020007 | 26/08/21 | 18:00 | Sala Eksperymentalna | 3 | 210 |
| 8 | 2020008 | 26/09/15 | 19:00 | Duża Scena | 2 | 230 |
| 9 | 2020009 | 26/10/30 | 18:45 | Mała Scena | 2 | 160 |
| 10 | 2020010 | 26/12/05 | 19:15 | Sala Nowa | 2 | 230 |

**- Przypadek użycia 4**. Widzowie o ponadprzeciętnych wydatkach

```
SELECT w.w_id, UPPER(w.Imię) || ' ' || UPPER(w.Nazwisko) widz, SUM(b.Cena) wydatki
FROM Widzowie w
JOIN Rezerwacje rz ON rz.Widzowie_w_id = w.w_id
JOIN Bilety b ON b.Rezerwacje_rz_id = rz.rz_id
GROUP BY w.w_id, w.Imię, w.Nazwisko
HAVING SUM(b.Cena) > (SELECT AVG(wydatki_na_widza)
                      FROM (SELECT SUM(b2.Cena) wydatki_na_widza
                            FROM Rezerwacje rz2
                            JOIN Bilety b2 ON b2.Rezerwacje_rz_id = rz2.rz_id
                            GROUP BY rz2.Widzowie_w_id)
)
ORDER BY wydatki DESC;
```

Opis: Zapytanie wyświetla widzów, których łączna kwota wydana na bilety jest wyższa niż średnia kwota wydana przez jednego widza

| | W_ID | WIDZ | WYDATKI |
|---|---|---|---|
| 1 | 1010002 | BEATA NOWICKA | 400 |
| 2 | 1010012 | LEON MAZUR | 310 |
| 3 | 1010006 | FILIP ZAJĄC | 310 |
| 4 | 1010001 | ADAM KOWAL | 240 |
| 5 | 1010011 | KINGA PAWLAK | 230 |
| 6 | 1010008 | HUBERT SZULC | 230 |
| 7 | 1010014 | NORBERT RYBAK | 200 |

**- Przypadek użycia 5**. Najbliższy termin dla każdego spektaklu

```
SELECT sp.sp_id, sp.Tytuł, (SELECT MIN(t.Data_spektaklu)
                    FROM Terminy t
                    JOIN Terminy_Spektakle ts ON ts.Terminy_t_id = t.t_id
                    WHERE ts.Spektakle_sp_id = sp.sp_id
                    AND t.Data_spektaklu >= TRUNC(SYSDATE))  najblizsza_data,
                    (SELECT MIN(t.Godzina)
                    FROM Terminy t
                    JOIN Terminy_Spektakle ts ON ts.Terminy_t_id = t.t_id
                    WHERE ts.Spektakle_sp_id = sp.sp_id
                    AND t.Data_spektaklu =
                       (SELECT MIN(t2.Data_spektaklu)
                       FROM Terminy t2
                       JOIN Terminy_Spektakle ts2 ON ts2.Terminy_t_id = t2.t_id
                       WHERE ts2.Spektakle_sp_id = sp.sp_id
                       AND t2.Data_spektaklu >= TRUNC(SYSDATE)
                          )
                       ) najblizsza_godzina
FROM Spektakle sp
WHERE (SELECT MIN(t2.Data_spektaklu)
      FROM Terminy t2
      JOIN Terminy_Spektakle ts2 ON ts2.Terminy_t_id = t2.t_id
      WHERE ts2.Spektakle_sp_id = sp.sp_id
      AND t2.Data_spektaklu >= TRUNC(SYSDATE)
      )IS NOT NULL
ORDER BY najblizsza_data NULLS LAST, najblizsza_godzina NULLS LAST, sp.Tytuł;
```

Opis: Zapytanie ma na celu wyświetlenie najbliższych terminów spektakli w teatrze - od dnia dzisiejszego

| | SP_ID | TYTUŁ | NAJBLIZSZA_DATA | NAJBLIZSZA_GODZINA |
|---|---|---|---|---|
| 1 | 2010002 | Dziady | 26/02/14 | 17:30 |
| 2 | 2010003 | Zemsta | 26/03/05 | 19:30 |
| 3 | 2010004 | Mistrz i Małgorzata | 26/04/12 | 18:30 |
| 4 | 2010005 | Wesele | 26/08/21 | 18:00 |

**- Przypadek użycia 6**. Analiza momentu zakupu biletu

```
SELECT rz.rz_id, w.Imię || ' ' || w.Nazwisko widz, t.Data_spektaklu, rz.Data_rezerwacji,
(t.Data_spektaklu - TRUNC(rz.Data_rezerwacji)) dni_przed,
  CASE
    WHEN (t.Data_spektaklu - TRUNC(rz.Data_rezerwacji)) <= 1 THEN 'Last minute (0-1)'
    WHEN (t.Data_spektaklu - TRUNC(rz.Data_rezerwacji)) <= 7 THEN 'Tydzień przed (2-7)'
    ELSE 'Wcześniej (8+)'
  END typ_zakupu
FROM Rezerwacje rz
JOIN Widzowie w ON w.w_id = rz.Widzowie_w_id
JOIN Bilety b ON b.Rezerwacje_rz_id = rz.rz_id
JOIN Terminy t ON t.t_id = b.Terminy_t_id
WHERE (t.Data_spektaklu - TRUNC(rz.Data_rezerwacji))>0
ORDER BY dni_przed ASC, t.Data_spektaklu;
```

Opis: Zapytanie oblicza, ile dni przed datą spektaklu widz dokonał rezerwacji biletu. Na podstawie tej daty klasyfikuje się zakup do jednej z kategorii: last minute, tydzień przed lub wcześniej.

| | RZ_ID | WIDZ | DATA_SPEKTAKLU | DATA_REZERWACJI | DNI_PRZED | TYP_ZAKUPU |
|---|---|---|---|---|---|---|
| 1 | 3020001 | Adam Kowal | 26/01/10 | 26/01/09 | 1 | Last minute (0-1) |
| 2 | 3020001 | Adam Kowal | 26/01/10 | 26/01/09 | 1 | Last minute (0-1) |
| 3 | 3020003 | Celina Borowska | 26/01/25 | 26/01/24 | 1 | Last minute (0-1) |
| 4 | 3020014 | Norbert Rybak | 26/02/14 | 26/02/13 | 1 | Last minute (0-1) |
| 5 | 3020014 | Norbert Rybak | 26/02/14 | 26/02/13 | 1 | Last minute (0-1) |
| 6 | 3020006 | Filip Zając | 26/04/12 | 26/04/11 | 1 | Last minute (0-1) |
| 7 | 3020006 | Filip Zając | 26/04/12 | 26/04/11 | 1 | Last minute (0-1) |
| 8 | 3020006 | Filip Zając | 26/04/12 | 26/04/11 | 1 | Last minute (0-1) |
| 9 | 3020009 | Iga Kopeć | 26/08/21 | 26/08/20 | 1 | Last minute (0-1) |
| 10 | 3020009 | Iga Kopeć | 26/08/21 | 26/08/20 | 1 | Last minute (0-1) |
| 11 | 3020008 | Hubert Szulc | 26/06/03 | 26/06/01 | 2 | Tydzień przed (2-7) |
| 12 | 3020013 | Marta Urban | 26/02/14 | 26/02/10 | 4 | Tydzień przed (2-7) |
| 13 | 3020013 | Marta Urban | 26/02/14 | 26/02/10 | 4 | Tydzień przed (2-7) |
| 14 | 3020005 | Eliza Kalinowska | 26/03/05 | 26/03/01 | 4 | Tydzień przed (2-7) |
| 15 | 3020005 | Eliza Kalinowska | 26/03/05 | 26/03/01 | 4 | Tydzień przed (2-7) |
| 16 | 3020004 | Dawid Bielecki | 26/02/14 | 26/02/01 | 13 | Wcześniej (8+) |
| 17 | 3020007 | Gosia Wrona | 26/06/03 | 26/05/20 | 14 | Wcześniej (8+) |
| 18 | 3020007 | Gosia Wrona | 26/06/03 | 26/05/20 | 14 | Wcześniej (8+) |
| 19 | 3020012 | Leon Mazur | 26/12/05 | 26/11/20 | 15 | Wcześniej (8+) |
| 20 | 3020001 | Adam Kowal | 26/01/25 | 26/01/09 | 16 | Wcześniej (8+) |
| 21 | 3020002 | Beata Nowicka | 26/01/10 | 25/12/20 | 21 | Wcześniej (8+) |
| 22 | 3020002 | Beata Nowicka | 26/01/10 | 25/12/20 | 21 | Wcześniej (8+) |
| 23 | 3020004 | Dawid Bielecki | 26/03/05 | 26/02/01 | 32 | Wcześniej (8+) |
| 24 | 3020002 | Beata Nowicka | 26/01/25 | 25/12/20 | 36 | Wcześniej (8+) |
| 25 | 3020002 | Beata Nowicka | 26/01/25 | 25/12/20 | 36 | Wcześniej (8+) |
| 26 | 3020008 | Hubert Szulc | 26/12/05 | 26/06/01 | 187 | Wcześniej (8+) |

**- Przypadek użycia 7**. Obliczenie czasu pozostałego do rozpoczęcia spektaklu

```
SELECT t.t_id, t.Data_spektaklu, t.Godzina, ROUND((TO_DATE(TO_CHAR(t.Data_spektaklu,
'YYYY-MM-DD') || ' ' || t.Godzina, 'YYYY-MM-DD HH24:MI') - SYSDATE) * 24, 2) godzin_do_startu
FROM Terminy t
WHERE ROUND((TO_DATE(TO_CHAR(t.Data_spektaklu, 'YYYY-MM-DD') || ' ' || t.Godzina,
'YYYY-MM-DD HH24:MI') - SYSDATE) * 24, 2) >0
AND t.Data_spektaklu <= ADD_MONTHS(TRUNC(SYSDATE), 3)
ORDER BY godzin_do_startu;
```

Opis: Zapytanie oblicza liczbę godzin pozostałych do rozpoczęcia każdego spektaklu odbywających się w ciągu najbliższych 3 miesięcy (Biorąc godzinę aktualną, więc o każdej godzinie w jakiej zapytanie jest wywoływane, wartości będą inne)

| | T_ID | DATA_SPEKTAKLU | GODZINA | GODZIN_DO_STARTU |
|---|---|---|---|---|
| 1 | 2020003 | 26/02/14 | 17:30 | 220,21 |
| 2 | 2020004 | 26/03/05 | 19:30 | 678,21 |
| 3 | 2020005 | 26/04/12 | 18:30 | 1589,21 |

**- Przypadek użycia 8**. Analiza czasu ról w spektaklu

```
SELECT sp.Tytuł, sp.Czas_trwania,(SELECT SUM(r.Czas_na_scenie)
                                  FROM Role r
                                  WHERE r.Spektakle_sp_id = sp.sp_id) suma_czasu_rol,
                                  (SELECT COUNT(*)
                                  FROM Role r
                                  WHERE r.Spektakle_sp_id = sp.sp_id) liczba_rol,
                                  ROUND(
                                  100 * (SELECT SUM(r.Czas_na_scenie)
                                  FROM Role r
                                  WHERE r.Spektakle_sp_id = sp.sp_id) / sp.Czas_trwania, 2)
proc_wzgledem_czasu
FROM Spektakle sp
ORDER BY proc_wzgledem_czasu DESC;
```

Opis: Dla każdego spektaklu obliczana jest suma czasu występowania wszystkich ról oraz jej procentowy udział względem całkowitego czasu trwania spektaklu. Dodatkowo prezentowana jest liczba ról przypisanych do danego spektaklu.

| | TYTUŁ | CZAS_TRWANIA | SUMA_CZASU_ROL | LICZBA_ROL | PROC_WZGLEDEM_CZASU |
|---|---|---|---|---|---|
| 1 | Zemsta | 110 | 110 | 3 | 100 |
| 2 | Mistrz i Małgorzata | 150 | 140 | 3 | 93,33 |
| 3 | Hamlet | 165 | 145 | 4 | 87,88 |
| 4 | Dziady | 190 | 155 | 4 | 81,58 |
| 5 | Wesele | 135 | 102 | 4 | 75,56 |

# 6. Pozostałe obiekty bazy danych

**- Sekwencer_1 (rezerwacje)**

```
CREATE SEQUENCE seq_rezerwacje
START WITH 3020001
INCREMENT BY 1
NOCACHE
NOCYCLE;
```

Sekwencer został utworzony w celu generowania identyfikatorów rezerwacji. Pierwsza generowana wartość to 3020001, a każda kolejna jest o jedna większa.

**- Sekwencer_2 (widzowie)**

```
CREATE SEQUENCE seq_widzowie
START WITH 1010001
INCREMENT BY 1
NOCACHE
NOCYCLE;
```

Sekwencer ten, podobnie jak wcześniejszy, został stworzony do generowania identyfikatorów widzów. Pierwszą jego wartością jest 1010001. Kolejnymi wartościami sekwencera jest wartość o jedna większa od poprzedniej.

**- Perspektywa_1 (repertuar)**

```
CREATE VIEW repertuar AS
SELECT t.Data_spektaklu, t.Godzina, s.Nazwa sala, sp.Tytuł tytul, r.Imię || ' ' || r.Nazwisko rezyser
FROM Terminy t JOIN Sale s ON s.s_id = t.Sale_s_id
   JOIN Terminy_Spektakle ts ON ts.Terminy_t_id = t.t_id
   JOIN Spektakle sp ON sp.sp_id = ts.Spektakle_sp_id
   JOIN Rezyserzy_Spektakle rs ON rs.Spektakle_sp_id = sp.sp_id
   JOIN Reżyserzy r ON r.r_id = rs.Reżyserzy_rz_id
WHERE t.Data_spektaklu >= DATE '2026-01-01'
ORDER BY t.Data_spektaklu, t.Godzina;
```

**- Perspektywa 2 (oblozenie_sali)**

```
CREATE VIEW oblozenie_sali AS
SELECT t.Data_spektaklu, t.Godzina, s.Nazwa sala, s.Liczba_miejsc, COUNT(b.b_id) sprzedane,
ROUND(COUNT(b.b_id) / s.Liczba_miejsc * 100, 2) oblozenie_proc
FROM Terminy t
JOIN Sale s ON s.s_id = t.Sale_s_id
LEFT JOIN Bilety b ON b.Terminy_t_id = t.t_id
GROUP BY t.Data_spektaklu, t.Godzina, s.Nazwa, s.Liczba_miejsc
HAVING COUNT(b.b_id) <= 0.01 * s.Liczba_miejsc
ORDER BY oblozenie_proc DESC;
```

**- Perspektywa 3 (przychod)**

```
CREATE VIEW przychod AS
SELECT t.t_id, t.Data_spektaklu, t.Godzina, s.Nazwa sala, NVL(x.liczba_biletow, 0) liczba_biletow,
NVL(x.przychod, 0) przychod
FROM Terminy t
JOIN Sale s ON s.s_id = t.Sale_s_id
LEFT JOIN (
  SELECT
    Terminy_t_id,
    COUNT(*) liczba_biletow,
    SUM(Cena) przychod
  FROM Bilety
  GROUP BY Terminy_t_id
) x ON x.Terminy_t_id = t.t_id
ORDER BY t.Data_spektaklu, t.Godzina;
```

## - Perspektywa 4 (wydatki)

```
CREATE VIEW wydatki AS
SELECT w.w_id, UPPER(w.Imię) || ' ' || UPPER(w.Nazwisko) widz, SUM(b.Cena) wydatki
FROM Widzowie w
JOIN Rezerwacje rz ON rz.Widzowie_w_id = w.w_id
JOIN Bilety b ON b.Rezerwacje_rz_id = rz.rz_id
GROUP BY w.w_id, w.Imię, w.Nazwisko
HAVING SUM(b.Cena) > (SELECT AVG(wydatki_na_widza)
                FROM (SELECT SUM(b2.Cena) wydatki_na_widza
                    FROM Rezerwacje rz2
                    JOIN Bilety b2 ON b2.Rezerwacje_rz_id = rz2.rz_id
                    GROUP BY rz2.Widzowie_w_id)
)
ORDER BY wydatki DESC;
```

## - Perspektywa 5 (najwczesniejszy_termin)

```
CREATE VIEW najwczesniejszy_termin AS
SELECT sp.sp_id, sp.Tytuł, (SELECT MIN(t.Data_spektaklu)
                    FROM Terminy t
                    JOIN Terminy_Spektakle ts ON ts.Terminy_t_id = t.t_id
                    WHERE ts.Spektakle_sp_id = sp.sp_id
                    AND t.Data_spektaklu >= TRUNC(SYSDATE))  najblizsza_data,
                    (SELECT MIN(t.Godzina)
                    FROM Terminy t
                    JOIN Terminy_Spektakle ts ON ts.Terminy_t_id = t.t_id
                    WHERE ts.Spektakle_sp_id = sp.sp_id
                    AND t.Data_spektaklu =
                       (SELECT MIN(t2.Data_spektaklu)
                       FROM Terminy t2
                       JOIN Terminy_Spektakle ts2 ON ts2.Terminy_t_id = t2.t_id
                       WHERE ts2.Spektakle_sp_id = sp.sp_id
                       AND t2.Data_spektaklu >= TRUNC(SYSDATE)
                          )
                       ) najblizsza_godzina
FROM Spektakle sp
WHERE (SELECT MIN(t2.Data_spektaklu)
      FROM Terminy t2
```

```
    JOIN Terminy_Spektakle ts2 ON ts2.Terminy_t_id = t2.t_id
    WHERE ts2.Spektakle_sp_id = sp.sp_id
    AND t2.Data_spektaklu >= TRUNC(SYSDATE)
    )IS NOT NULL
ORDER BY najblizsza_data NULLS LAST, najblizsza_godzina NULLS LAST, sp.Tytuł;
```

## - Perspektywa 6 (ile_dni_przed)

```
CREATE VIEW ile_dni_przed AS
SELECT rz.rz_id, w.Imię || ' ' || w.Nazwisko widz, t.Data_spektaklu, rz.Data_rezerwacji,
(t.Data_spektaklu - TRUNC(rz.Data_rezerwacji)) dni_przed,
  CASE
    WHEN (t.Data_spektaklu - TRUNC(rz.Data_rezerwacji)) <= 1 THEN 'Last minute (0-1)'
    WHEN (t.Data_spektaklu - TRUNC(rz.Data_rezerwacji)) <= 7 THEN 'Tydzień przed (2-7)'
    ELSE 'Wcześniej (8+)'
  END typ_zakupu
FROM Rezerwacje rz
JOIN Widzowie w ON w.w_id = rz.Widzowie_w_id
JOIN Bilety b ON b.Rezerwacje_rz_id = rz.rz_id
JOIN Terminy t ON t.t_id = b.Terminy_t_id
WHERE (t.Data_spektaklu - TRUNC(rz.Data_rezerwacji))>0
ORDER BY dni_przed ASC, t.Data_spektaklu;
```

## - Perspektywa 7 (godziny_do_spektaklu)

```
CREATE VIEW godziny_do_spektaklu AS
SELECT t.t_id, t.Data_spektaklu, t.Godzina, ROUND((TO_DATE(TO_CHAR(t.Data_spektaklu,
'YYYY-MM-DD') || ' ' || t.Godzina, 'YYYY-MM-DD HH24:MI') - SYSDATE) * 24, 2) godzin_do_startu
FROM Terminy t
WHERE ROUND((TO_DATE(TO_CHAR(t.Data_spektaklu, 'YYYY-MM-DD') || ' ' || t.Godzina,
'YYYY-MM-DD HH24:MI') - SYSDATE) * 24, 2) >0
AND t.Data_spektaklu <= ADD_MONTHS(TRUNC(SYSDATE), 3)
ORDER BY godzin_do_startu;
```

## - Perspektywa 8 (czas_roli)

```
CREATE VIEW czas_roli AS
SELECT sp.Tytuł, sp.Czas_trwania,(SELECT SUM(r.Czas_na_scenie)
                                  FROM Role r
                                  WHERE r.Spektakle_sp_id = sp.sp_id) suma_czasu_rol,
                                  (SELECT COUNT(*)
                                  FROM Role r
                                  WHERE r.Spektakle_sp_id = sp.sp_id) liczba_rol,
                                  ROUND(
                                  100 * (SELECT SUM(r.Czas_na_scenie)
                                  FROM Role r
                                  WHERE r.Spektakle_sp_id = sp.sp_id) / sp.Czas_trwania, 2)
proc_wzgledem_czasu
FROM Spektakle sp
ORDER BY proc_wzgledem_czasu DESC;
```